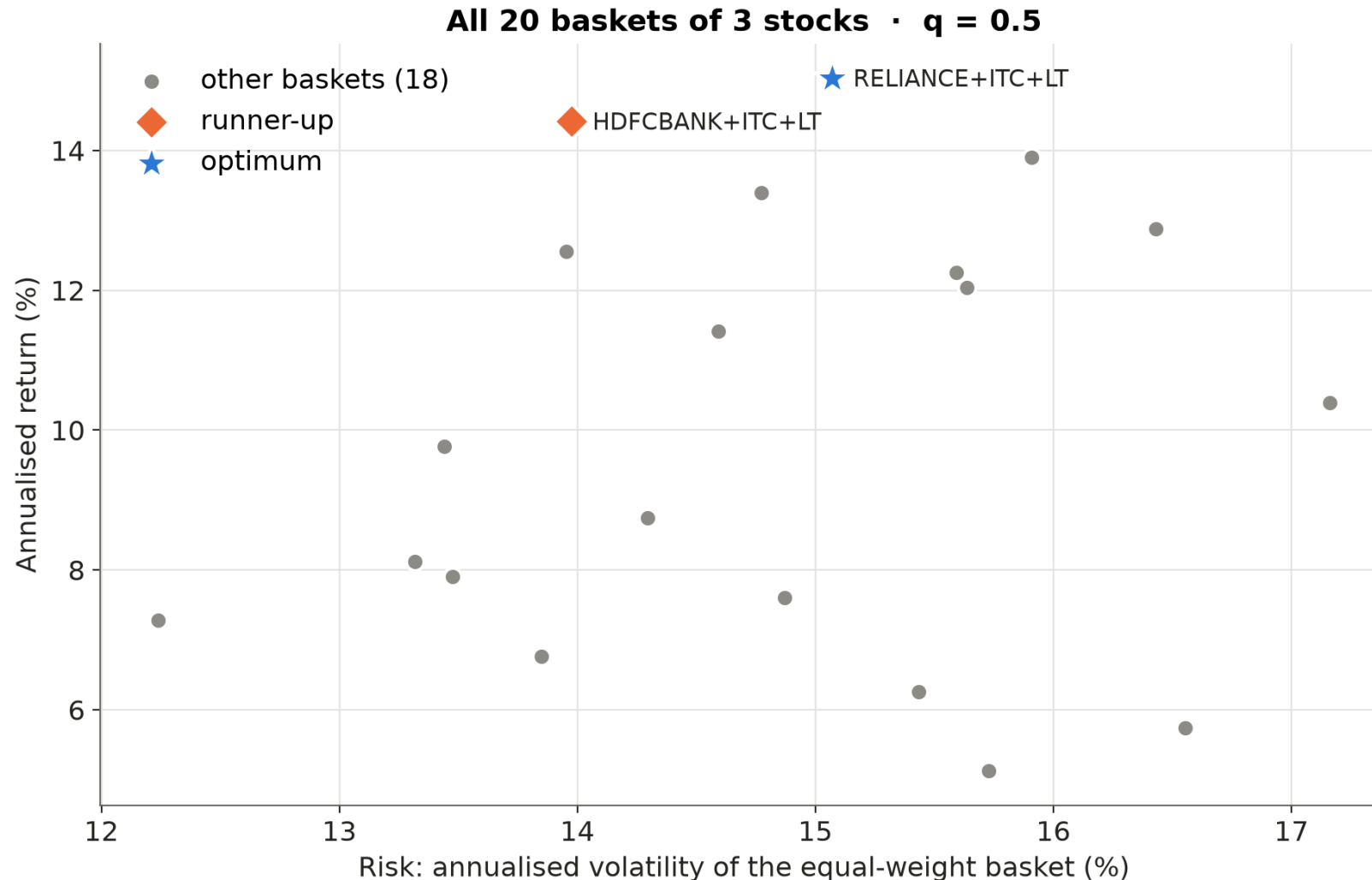


Picking the best 3 of 6 stocks, on a quantum computer

Robo-advisor model baskets · NSE large caps · return $\uparrow$ risk $\downarrow$



All $C(6,3) = 20$ baskets on the risk/return plane.

Basket count grows as $C(n,k)$:
~185 000 for 10-of-20.

Goal: find the best basket for a client's risk aversion q .

Put the constraint in the circuit, not in the cost

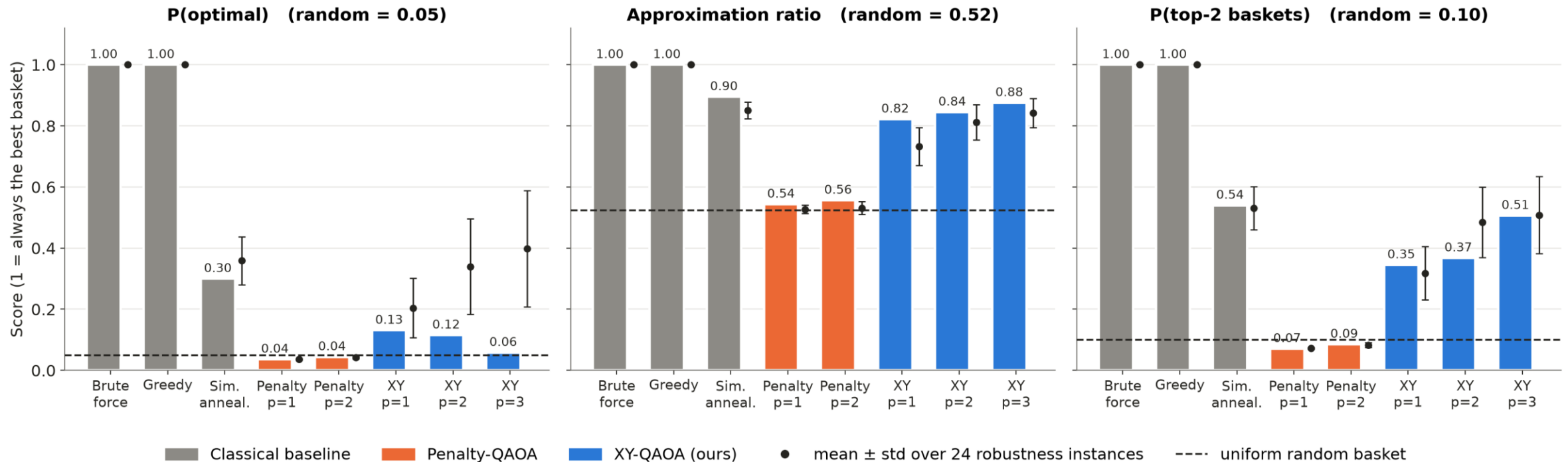
Penalty QAOA (X mixer)

Searches all 64 bit strings, 44 of them invalid.
→ finds the best basket at chance level

XY-QAOA (Dicke state + XY mixer)

Only ever visits the 20 valid baskets.
→ every shot is a real portfolio

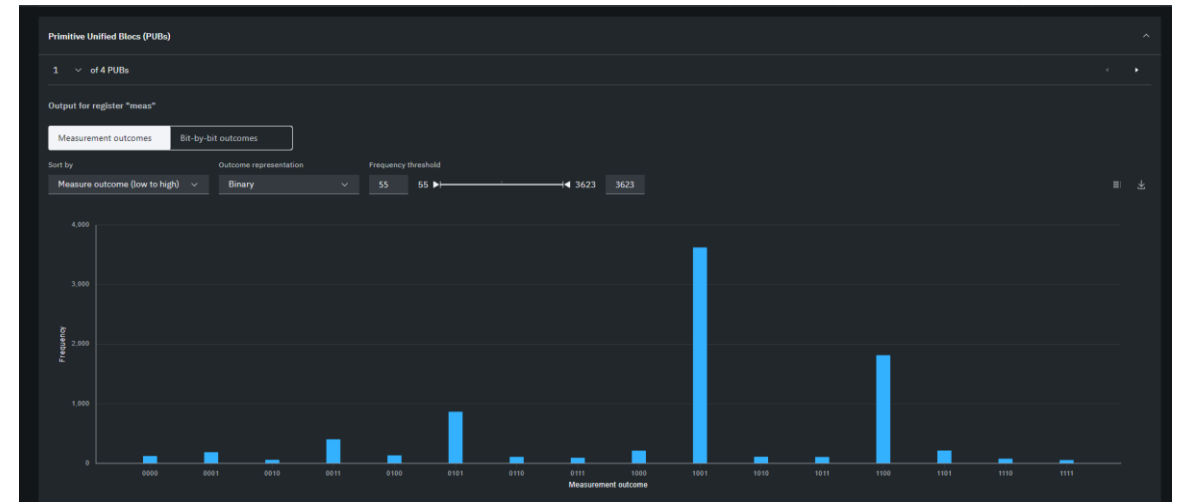
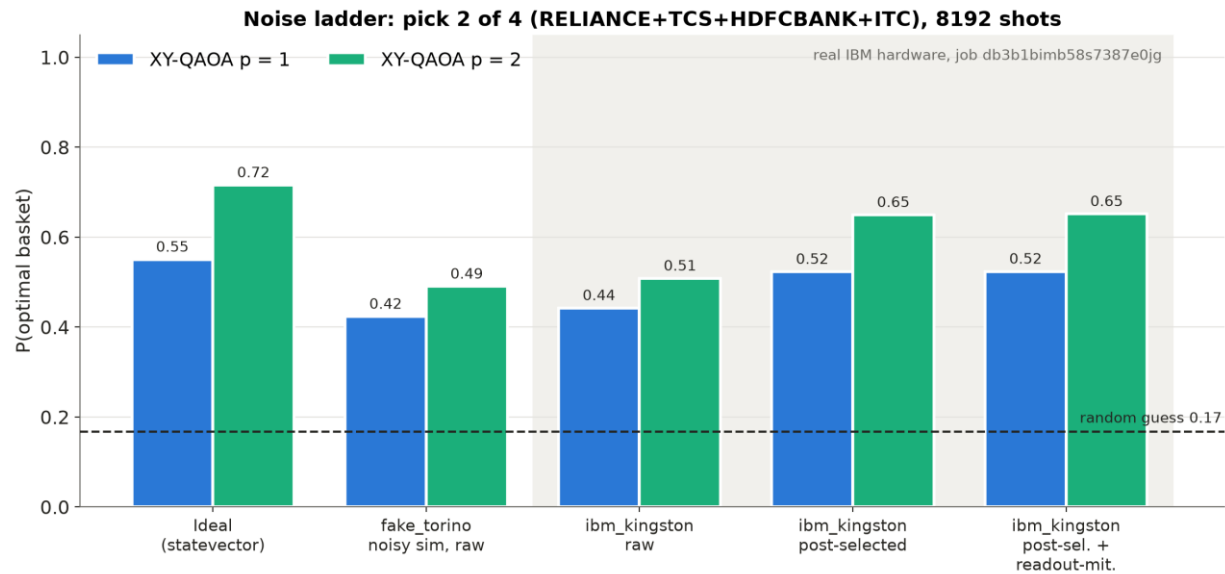
Headline instance: pick 3 of 6 NSE stocks (RELIANCE, TCS, HDFCBANK, INFY, ITC, LT), $q = 0.5$ · ideal simulator



~7× chance (P(optimal) 0.34 vs 0.05, 24 instances) · 100% valid · penalty-QAOA = chance (0.04, 75% valid)

Real hardware (ibm_kingston) and honest limits

P(optimal) 0.51 raw $\rightarrow$ 0.65 post-selected vs 0.17 random · job db3b1bimb58s7387e0jg



Real ibm_kingston output — tallest bar 1001 = optimal basket (RELIANCE+ITC), second 1100 = runner-up

- Symmetry post-selection = free error detection
- Classical brute force is still faster at this size: no quantum advantage claimed
- Next: more business constraints as circuit symmetries

Repo: github.com/achintya007-arch/qfolio

Report: achintya007-arch.github.io/qfolio